

Security Audit

Textile (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	9
Intended Smart Contract Functions	10
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	24
Conclusion	25
Our Methodology	27
Disclaimers	29
About Hashlock	30

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Textile team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Textile FX is a non-custodial, on-chain foreign exchange platform that enables users to swap local-currency stablecoins such as cNGN, representing the Nigerian naira, for USD stablecoins like USDT. Operating across supported blockchain networks including BNB Smart Chain, the platform provides instant settlement through firm quotes or limit orders, with rates and fees transparently displayed on-chain.

Project Name: Textile

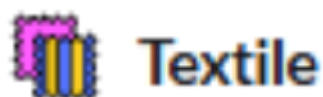
Project Type: DeFi, Tokenized Vault, Asset Management, Yield, Signed NAV Settlement, On-chain Trade Execution

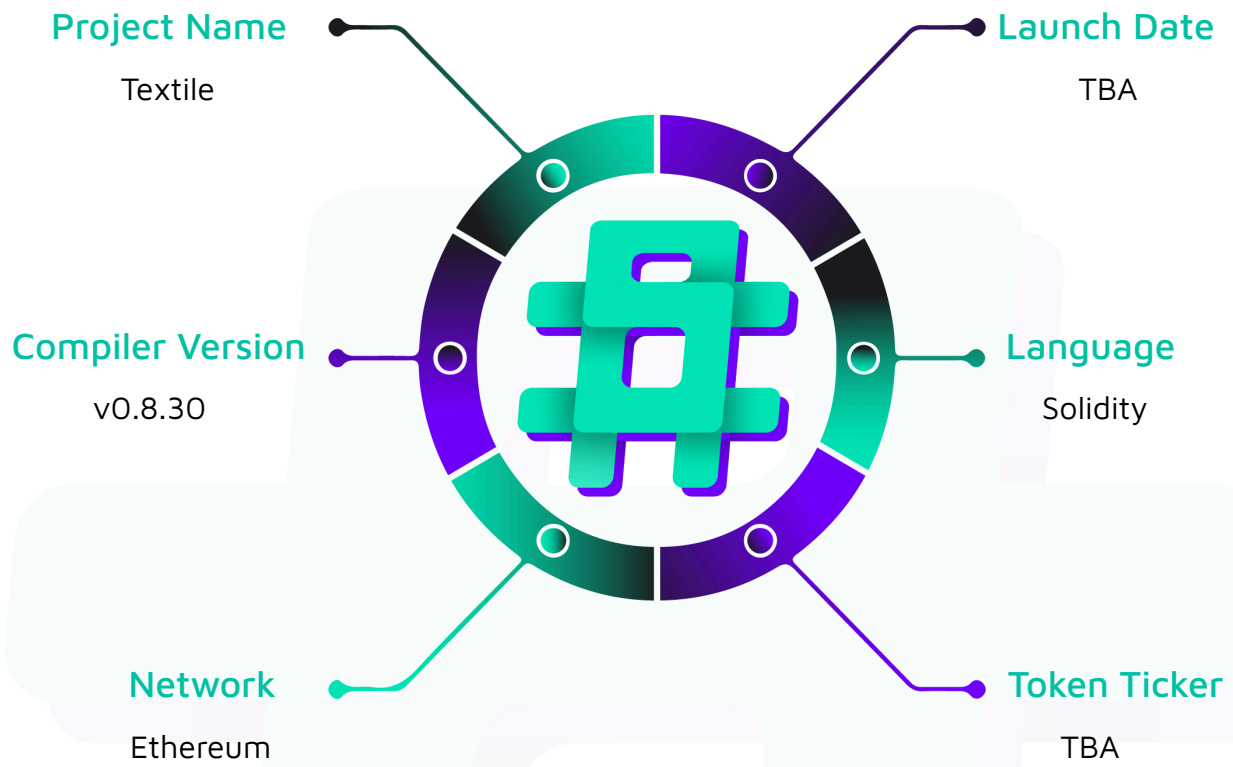
Compiler Version: 0.8.30

Website:

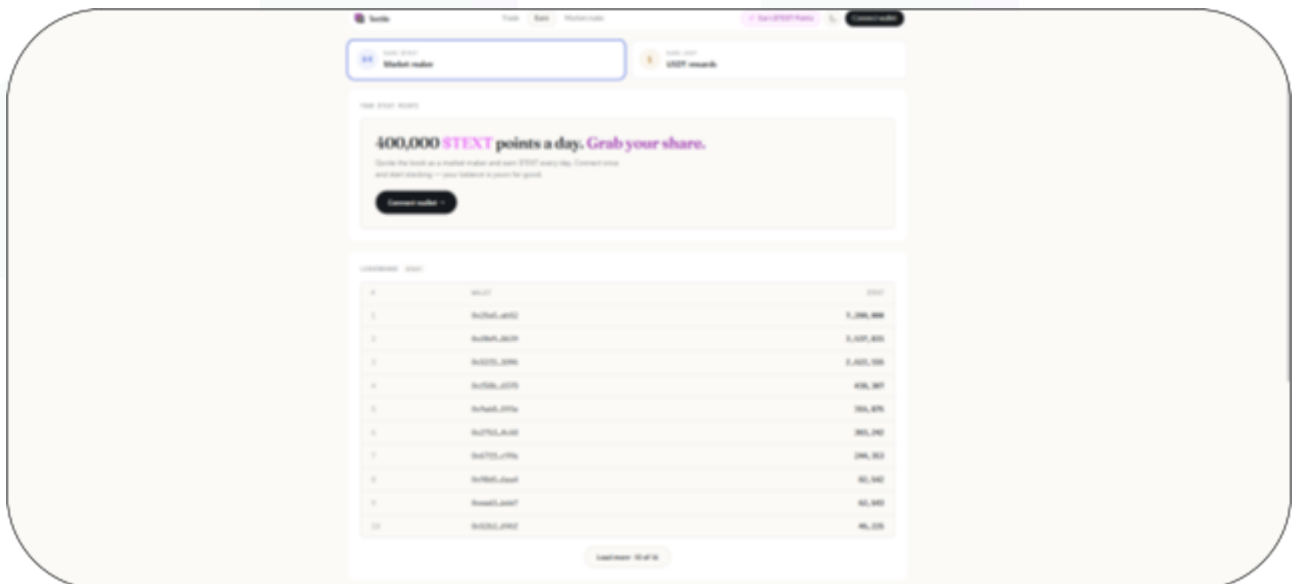
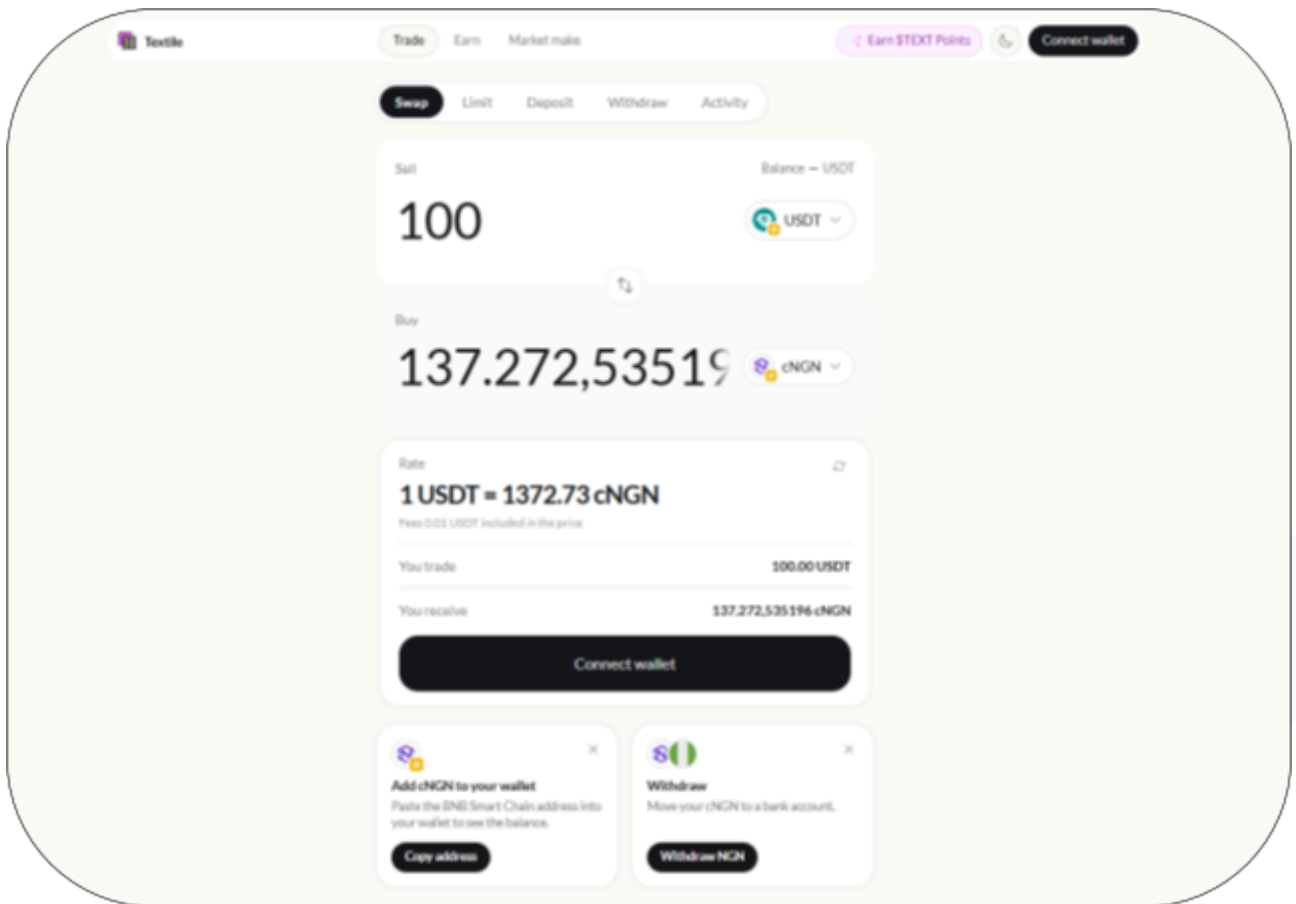
<https://app.textilecredit.com/s/swap?sell=USDT&buy=cNGN&chainId=56&amount=100>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Textile project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Textile Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	September, 2026
Contract 1	OperatorVault.sol
Contract 2	OperatorVaultFactory.sol
Contract 3	VaultOrderExecutor.sol
Contract 4	SellFirstFeeController.sol
Contract 5	PreferredFillerValidation.sol
Contract 6	FillerConstants.sol
Contract 7	VaultDeployer.sol
Contract 8	AaveV3YieldAdapter.sol
Library 1	VaultPolicy.sol
Library 2	VaultLib.sol
Library 3	VaultTypes.sol
Library 4	VaultErrors.sol
Audited GitHub Commit Hash	06d976ce013d24f97cea7b75e14d2ed68a37bf6f
Fix Review GitHub Commit Hash	8ea7aaa348112b629b2ce5d640571be8679a86f4

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.



Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of OpenZeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

3 Medium severity vulnerabilities

2 Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
OperatorVault.sol - Accepts settlement-asset or corridor-asset deposits and settles them in aggregate epochs. - Issues transferable ERC-20 shares and processes pro-rata two-asset redemptions. - Authorizes constrained UniswapX orders through ERC-1271 and optionally deploys idle settlement assets into Aave.	Contract achieves this functionality.
OperatorVaultFactory.sol - Deploys immutable vaults with operator-selected risk parameters. - Clones and binds one optional yield adapter per vault. - Indexes vaults by operator, asset pair, and version and rekeys them after admin transfer.	Contract achieves this functionality.
VaultOrderExecutor.sol - Recalls settlement liquidity before the reactor's Permit2 pull. - Re-enforces preferred-filler binding at the wrapper boundary. - Executes the order and returns all ERC-20 balances to the caller in the same transaction.	Contract achieves this functionality.
SellFirstFeeController.sol - Aggregates duplicate output tokens before calculating fees. - Charges an immutable, deployment-specific output fee to an immutable recipient.	Contract achieves this functionality.

- Prevents orders whose calculated fee would round to zero from executing fee-free.	
PreferredFillerValidation.sol - Restricts an order to a bounded set of preferred fillers during the exclusivity window. - Opens the order after the signed exclusivity deadline. - Accepts validation calls only from the order's reactor.	Contract achieves this functionality.
AaveV3YieldAdapter.sol - Supplies and recalls idle settlement assets through Aave V3. - Tracks the rebasing aToken position and transfers emergency in-kind claims. - Sends force-transferred underlying or junk tokens back to the vault.	Contract achieves this functionality.
VaultLib.sol and VaultPolicy.sol - Implement share conversion, NAV hashing, Permit2 digest construction, and order-policy checks. - Validate deployment configuration and administer roles. - Coordinate yield reserves, token recovery, and settlement attestations.	Libraries achieve this functionality.

Code Quality

This audit scope involves the smart contracts of the Textile project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Textile project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] contracts/v3/filler/vault/VaultOrderExecutor.sol#fill - Forced reactor ETH persistently DoSes VaultOrderExecutor

Description

The reactor refunds its entire residual ETH balance to its caller, but `VaultOrderExecutor` cannot receive ETH.

Vulnerability Details

`VaultOrderExecutor.fill()` calls the reactor without sending ETH:

```
outputToken.forceApprove(address(reactor), totalOutput);
reactor.execute(signedOrder);
outputToken.forceApprove(address(reactor), 0);
```

The reactor accepts direct ETH transfers and refunds any remaining balance to `msg.sender`, which is the executor:

```
if (address(this).balance > 0) {
    CurrencyLibrary.transferNative(msg.sender, address(this).balance);
}

receive() external payable {}
```

Because the executor has no payable `receive()` or fallback, one wei sent to the reactor makes the refund fail and reverts every executor fill. The balance remains because the transaction reverts. It can be deliberately cleared through a successful direct reactor fill, but the attacker can immediately deposit another wei.

Impact

Any account can cheaply and repeatedly block all fills routed through `VaultOrderExecutor`, potentially preventing time-sensitive rebalancing or hedging.

Recommendation

Add a payable receiver to the executor and a restricted ETH recovery function:

```
receive() external payable {}
```

Status

Resolved

Note

This finding was identified during the consultancy phase and fixed in commit 8f7dd3d, before the audit commit.

[M-02]**contracts/v3/filler/vault/OperatorVault.sol#settleRedeemEmergencyInKind**

- A UniswapX filler can settle redemptions against transiently depleted balances

Description

Emergency redemption settlement prices the epoch from the vault's current balances. The pinned UniswapX reactor transfers the vault's input tokens away before it calls the filler, so a listed filler can snapshot that hole.

Vulnerability Details

`executeWithCallback` pulls the order input, then calls the filler, and only afterwards delivers the output:

```
_prepare(resolvedOrders);
IReactorCallback(msg.sender).reactorCallback(resolvedOrders, callbackData);
_fill(resolvedOrders);
```

`_prepare` transfers the vault's input inside that first step. From the callback, the filler calls the permissionless emergency entry point. It pauses, then reserves a pro-rata share of whatever balances are still on hand:

```
function settleRedeemEmergencyInKind(uint256 epochId) external override nonReentrant
{
    Epoch storage epoch = _closedEpoch(epochId, false);
    _requireElapsed(epoch.closedAt, emergencyExitTimeout);
    lastFeeCheckpoint = block.timestamp;
    if (!paused) _pause();
    // ...
    uint256 freeSettlementAssets = _liquidSettlement();
    uint256 freeCorridorAssets = _freeCorridor();
    uint256 settlementOut = Math.mulDiv(freeSettlementAssets, shares, supply);
    uint256 corridorOut = Math.mulDiv(freeCorridorAssets, shares, supply);
```

Those reads are the raw balances minus pending deposits and reserves:

```
return settlementAsset.balanceOf(address(this)) - pendingSettlement -
reservedSettlement;
```

```
return corridorAsset.balanceOf(address(this)) - reservedCorridor - pendingCorridor;
```

The pause makes later `isValidSignature` checks fail, and `nonReentrant` is set only once this call enters the vault. Neither applies to the order already inside the reactor: its signature was checked in `_prepare`, and the reactor call began outside the vault. When the callback returns, `_fill` still delivers the output. The settled redeemers don't receive it.

With 100 settlement tokens and 900 corridor tokens in the vault, half the shares queued, and a signed corridor sale of 900, the callback sees 100 settlement and 0 corridor. The queued half is reserved about 50 settlement and no corridor. The output of 900 settlement arrives afterwards and accrues to the remaining shares, which then represent about 950.

Impact

Queued redeemers can lose most of the value represented by their shares. That value stays with the shareholders who remain, so a listed filler who also holds shares profits from an otherwise fair trade. The same split applies to any balance-based settlement that can be entered from the filler callback.

Recommendation

Consider not snapshotting balances in the same transaction as an in-flight fill. Open settlement in one transaction, which disables new order validation, and take the balance snapshot only in a later transaction. Alternatively, route every trade through a vault entry point that holds an in-flight lock, or keep trading inventory in an escrow whose receivable is still counted. A `nonReentrant` modifier on the settlement function doesn't cover a callback that enters the vault for the first time.

Status

Resolved

[M-03] contracts/v3/filler/vault/OperatorVault.sol#processDepositEpoch - Aggregate deposit accounting lets the last claimant steal another depositor's allocation

Description

Deposit settlement only requires the epoch to mint a nonzero number of shares. Each depositor is paid later by integer division, and the last claimant receives whatever shares the earlier divisions dropped.

Vulnerability Details

An epoch with existing supply mints from the aggregate deposit at the attested price, and reverts only when that total is zero:

```
uint256 shares = VaultLib.convertToShares(value, supply, supply == 0 ? 0 :
attestedNav, Math.Rounding.Floor);
if (shares == 0) revert VaultErrors.ZeroAmount();
_mint(address(this), shares);
```

`convertToShares` is $\text{assets} * (\text{supply} + 1) / (\text{totalAssets} + 1)$, rounded down. A later claim divides those shares by the depositor's fraction of the epoch. Anyone except the last depositor rounds down; the last depositor takes the remainder:

```
if (claimUnits == remainingUnits) return remainingOut;
return Math.mulDiv(remainingOut, claimUnits, remainingUnits);
```

The empty-vault donation guard does not apply. Supply is already nonzero, and the epoch itself mints one share. With one share outstanding and 200 USDC of assets, two 100 USDC deposits in the same epoch mint:

```
floor(200e6 * (1 + 1) / (200e6 + 1)) = 1
```

The first claim of 100 against 200 receives $\text{floor}(1 * 100 / 200) = 0$. The second claim is the last unit and receives the remaining share. The vault then holds 400 USDC against two shares. The attacker, who kept the original share and claims second, owns both and can redeem about 400 USDC. The roughly 200 USDC donation returns inside the original share. The gain is the victim's 100 USDC deposit.

This state is reachable with the default 6-decimal settlement token and the 100-token minimum deposit. The attacker redeems down to one share, donates assets until that share is worth about 200 USDC, and joins the victim's deposit epoch.

Impact

A depositor can lose an entire accepted deposit to a later claimant in the same epoch. The attacker needs the residual share supply, recoverable donation liquidity, one deposit in that epoch, and a claim that lands after the victim's.

Recommendation

At settlement, require enough shares for every minimum-sized deposit, and refund or roll the epoch forward otherwise:

```
if (shares < Math.ceilDiv(assets, minDepositAssets)) revert VaultErrors.ZeroAmount();
```

Every accepted nonzero deposit must then have a nonzero share entitlement. Consider not giving another depositor's rounded-away shares to whichever controller claims last. A preview at `requestDeposit` is not enough, because a donation can land after the request and before settlement.

Status

Resolved

Low

[L-01] `contracts/v3/filler/vault/libraries/VaultPolicy.sol#payYield` -

Yield-token aliasing can expose corridor inventory to yield claimants

Description

Vault configuration does not require the adapter yield token to differ from both working assets.

Vulnerability Details

Configuration validates only that the settlement and corridor assets differ:

```
if (address(cfg.settlementAsset) == address(cfg.corridorAsset)) {  
    revert VaultErrors.InvalidPair();  
}
```

The adapter-provided token is accepted without checking whether it aliases either asset:

```
yieldToken = cfg.yieldAdapter == address(0)  
    ? address(0)  
    : VaultPolicy.bindYieldAdapter(cfg.yieldAdapter, cfg.settlementAsset);
```

Yield claims are calculated from the vault's complete balance of that token:

```
uint256 yieldOut =  
    VaultLib.proRataWithResidue(weight, outstanding,  
    IERC20(yieldToken).balanceOf(address(this)));  
IERC20(yieldToken).safeTransfer(receiver, yieldOut);
```

If `yieldToken == corridorAsset`, this balance includes ordinary corridor liquidity, pending deposits, and reserved payouts. A final yield claimant can therefore receive unrelated corridor inventory. With the canonical Aave adapter, settlement-token aliasing is unlikely, but corridor-token aliasing remains configurable.

Impact

A misconfigured vault can transfer working corridor inventory belonging to other shareholders or claimants to emergency-yield claimants.

Recommendation

Reject working-asset aliases after binding the adapter:

```
if (yieldToken == address(settlementAsset) || yieldToken == address(corridorAsset)) {  
    revert YieldTokenAliasesWorkingAsset();  
}
```

Yield claims should additionally use tracked entitlement balances rather than the token's undifferentiated full vault balance.

Status

Resolved

[L-02] contracts/v3/filler/vault/OperatorVault.sol#processDepositEpoch -

One key alone moves nothing is false for LP settlement

Description

The protocol's documentation states that trading orders require exactly two signatures, but LP deposit and redemption settlement is authenticated only by the risk signer.

Vulnerability Details

The two-signature requirement applies to ERC-1271 order validation:

```
if (!VaultLib.isSigner(src.strategySigner(), hash, operatorSig)) return
VaultLib.ERC1271_FAIL;
if (!VaultLib.isSigner(src.riskSigner(), hash, riskSig)) return
VaultLib.ERC1271_FAIL;
```

Deposit and redemption settlement instead call `verifyAttestation()` with only the risk signer:

```
uint256 price =
    VaultPolicy.verifyAttestation(attestation, signature, epochId, address(this),
    _roles.riskSigner);
```

The attestation verification performs only that single signer check:

```
bytes32 digest = VaultLib.attestationDigest(att, vault, block.chainid);
if (!VaultLib.isSigner(riskSigner, digest, signature)) {
    revert VaultErrors.InvalidAttestation();
}
```

That signer controls values used to mint deposit shares, calculate redemption reserves, burn shares, and update settled NAV. Documentation claiming broadly that one key cannot move value therefore understates the risk signer's authority.

Impact

Users and integrators may rely on an incorrect one-key fault-tolerance assumption when evaluating LP settlement and recovery risk.

Recommendation

Require independent authorization for settlement attestations or explicitly disclose that the risk signer has unilateral authority over LP pricing and redemption snapshots.

Status

Resolved

Note

This finding was identified during the consultancy phase and fixed in commit 8f7dd3d, before the audit commit.

Centralisation

The Textile project values community-centered and transparency over centralisation.

Textile emphasizes a community-centric approach while maintaining transparency. Although some critical functions remain centralized for enhanced security and functionality, the project strives to balance decentralization with internal team accountability. This blend ensures operational security while keeping the protocol aligned with decentralized values.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Textile project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.